




University Institute of Computing
Chandigarh University, Gharuan, Mohali

P R O J E C T R E P O R T

O N

Student Result Management System

Subject: - Object-Oriented Programming (C++)
Subject Code: - 25CAH-201

SUBMITTED BY:

Name: Somya Gupta **UID: 25BCD10029**

Branch: BCA(Data Science) **Group: B, Section: BCD-3**

Semester: 03

SUBMITTED TO:

Name: Mr. Abhinay Sood **D.O. SUBMISSION: _____**

Designation: TT **Sign: _____**

INDEX

Sr. No.	Content
01	Title Page
02	Index
03	Introduction about the Topic
04	Objectives
05	Data Description
06	Tools and Technologies
07	Code with Explanation
08	Output
09	Conclusion
10	Learning Outcomes
11	GitHub Link: https://github.com/Somya-ds/Student-Result-Management-System

I. Introduction about the Topic

Student Result Management Systems

A Student Result Management System is a software application that stores the details of students, records the marks they obtain in different subjects, and converts those marks into meaningful results such as total marks, percentage and grade. Every school and college has to perform this work at the end of each term, and the quality of the result directly affects the student's academic standing, scholarships and future opportunities.

Importance of Managing Academic Records

Academic records are among the most important documents of an institution. They are needed to prepare report cards, to compare the performance of students, to identify students who need extra support and to answer queries long after a semester has ended. If these records are incomplete, inconsistent or wrongly calculated, the effect on the student can be serious. Therefore, records must be stored in an organised way and the calculations based on them must always follow the same rules.

Problems with Manual Result Management

When results are maintained on paper or in loosely organised registers, several problems appear:

- **Calculation errors:** totals and percentages are added by hand, so a small slip changes the final grade.
- **Time consumption:** searching for one student among many pages, or correcting a record, takes a lot of time.
- **Inconsistency:** different people may apply the grade boundaries differently.
- **Difficult updates:** a corrected mark requires rewriting totals, percentages and grades by hand.
- **No quick comparison or analysis:** comparing two students, or checking how a change in one subject affects the overall result, needs separate manual calculations.

Need for a Computerised System

A computerised result management system removes most of these problems. Once the rules for percentage and grade are written in a program, they are applied identically to every student, calculations are instantaneous, and records can be searched, corrected or removed in a few steps. Even a small console application can therefore make the process faster and far less error-prone than a manual register.

Overview of the Developed Application

The project "Student Result Management System" is a console-based application written in C++. It is menu-driven and offers ten options: add a student, display all students, search a student, update student details, update marks, generate a performance report, delete a student, run the What-If Grade Calculator, compare two students, and exit. The program can hold up to 50 students at a time, and each student can have up to 5 subjects with marks between 0 and 100.

How the Application Works

When the program starts, an array of 50 Student objects is created and a counter keeps track of how many are in use. The main menu is displayed repeatedly inside a do-while loop, and a switch statement runs the function that matches the user's choice. Each student is identified by a unique ID. The ID is used to search the array, and the matching object is then used to display, update, delete, simulate or compare the record. Percentage and grade are not typed in by the user; they are calculated automatically from the marks every time they are needed.

Importance of C++ and Object-Oriented Programming

C++ was chosen because it directly supports the object-oriented concepts studied in this subject. The common details of a person (ID and name) are placed in a base class, and the Student class inherits them and adds academic data and behaviour. Data is kept private or protected, and is changed only through member functions, which is encapsulation. Virtual functions allow the same call to behave differently for different class types. Using these concepts makes the program better organised, easier to understand and easier to extend than a single long procedural program.

Additional Unique Feature

Besides the standard operations, the project includes a **What-If Grade Calculator (Academic Performance Simulator)**. It lets the user pick a registered student and a subject, enter hypothetical marks, and immediately see the predicted percentage and grade next to the original ones, without changing the stored marks. It turns the system from a simple record keeper into a small planning tool; it is explained in detail in Section V.

II. Objectives

The main objectives of this project are listed below, together with the part of the program that fulfils each of them.

1. **Managing student information:** To store the ID, name, course and semester of each student using objects of the Student class, which inherits ID and name from the Person class.
2. **Adding and displaying student records:** To add new students (Menu option 1) with validation of the ID, semester, number of subjects and marks, and to display all stored students (Menu option 2).
3. **Searching for students using their IDs:** To locate any student quickly through the searchStudent() function, which scans the array and returns the position of the matching ID.
4. **Updating student details:** To change the name, course and semester of an existing student (Menu option 4) through the updateDetails() member function.
5. **Managing subject marks:** To store up to five subjects and their marks per student and to correct a mark later (Menu option 5) using updateMarks().
6. **Calculating total marks and percentage:** To add the marks of all subjects and divide by the number of subjects automatically in percentage().
7. **Automatically generating grades:** To convert the percentage to one of the grades A+, A, B, C, D or F through the overloaded grade() functions.
8. **Generating individual performance reports:** To print the complete result of one student (details, subject-wise marks, percentage and grade) with showResult() (Menu options 3 and 6).
9. **Comparing the academic performance of two students:** To compare the percentages of two students with the friend function compare() (Menu option 9).
10. **Providing an academic performance simulation:** To let a student see how hypothetical marks in one subject would change the overall percentage and grade, using the What-If Grade Calculator (Menu option 8).

In addition, the project aims to apply the OOPS concepts taught in the course (classes, objects, constructors, destructors, inheritance, function overloading, function overriding, virtual functions, pointers and friend functions) in one working application.

III. Data Description

The application works with two groups of data: personal and course information about the student, and academic information about the student's marks. All data is entered by the user while the program is running and is held in memory.

Student Information

Table 1: Attributes of a student record

Data Item	C++ Type	Stored In	Description
Student ID	string	Person (protected)	Unique identifier used to search, update, delete, simulate and compare.
Student Name	string	Person (protected)	Name of the student (a single word, because input is read with cin >>).
Course	string	Student (private)	Course of the student (a single word).
Semester	int	Student (private)	Semester number; accepted values are 1 to 12.

Academic Information

Table 2: Academic data maintained for each student

Data Item	C++ Type	Stored In	Description
Subject Names	string Subject[5]	Student (private)	Names of up to five subjects.
Subject Marks	int Marks[5]	Student (private)	Marks in each subject, accepted from 0 to 100.
Subject Count	int count	Student (private)	Number of subjects actually stored (0 to 5).
Total Marks	calculated	Local variable	Sum of the marks, calculated inside percentage(); not stored.
Percentage	float	Returned value	Total marks divided by count; calculated when required.
Grade	string	Returned value	Letter grade generated from the percentage.

Use of Arrays and Objects

Each student is represented by one object of the Student class. In main(), an array of 50 objects, **Student s[50]**, holds all the records, and the integer **n** records how many positions are in use. Inside every object two parallel arrays, **Subject[5]** and **Marks[5]**, hold the subject names and marks, with the same index referring to the same subject; the variable **count** shows how many entries are filled. Because the data members are private or protected, other parts of the program can use them only through member functions such as addStudent(), addSubject(), updateMarks() and percentage().

Grade Calculation System

The percentage is the total of all subject marks divided by the number of subjects (each subject is out of 100). The grade is then decided by the following conditions, exactly as written in the grade() function:

Table 3: Grade boundaries used by the system

Percentage	Grade
90 and above	A+
80 to below 90	A
70 to below 80	B
60 to below 70	C
50 to below 60	D
Below 50	F

Limitations of the Current Implementation

- Records exist only while the program is running. They are **not saved permanently**: closing the application erases all data, and every run starts with an empty list.
- The project does **not** use a database or file handling.
- At most 50 students and 5 subjects per student can be stored.
- Name, course and subject names are read with cin >>, so they must be a single word without spaces.
- Only the range of numeric values is validated; entering text where a number is expected is not handled.
- If an invalid mark is entered while adding subjects, the program prints "Invalid marks! Student not added." and stops asking for further subjects, but the final check in Menu option 1 is always true, so the student is still counted with the subjects entered so far. This is a known limitation of the current code and a point for improvement.

IV. Tools and Technologies

Only the tools and language features actually used in the supplied source code are listed here.

Table 4: Tools and technologies used in the project

Tool / Concept	Details
Programming Language	C++, a compiled language that supports both procedural and object-oriented programming.
Compiler / IDE	[Insert compiler and IDE used, e.g. g++ / VS Code]. The output screenshots show the program compiled to main.exe and run from a Windows PowerShell terminal.
Header: iostream	Provides cin and cout for keyboard input and screen output.
Header: string	Provides the string class for IDs, names, courses and subject names.
Arrays	Student s[50], string Subject[5] and int Marks[5] store the records, subject names and marks.
Classes and Objects	Person and Student classes; each student record is an object.
Constructors and Destructors	Person() and Student() initialise the data; Person has a virtual destructor ~Person().
Inheritance	Student publicly inherits from Person.
Function Overloading	Two grade() functions: grade(float p) and grade().

Tool / Concept	Details
Function Overriding	Student::display() overrides Person::display() (marked with override).
Virtual Functions	display() and the destructor are virtual in Person.
Pointers	A Person* pointer is used in “Display All Students” to call display() polymorphically.
Friend Functions	compare(Student, Student) is a friend of Student and can read the students’ private and protected data.

Why C++ Was Selected

C++ was selected because the project is meant to demonstrate Object-Oriented Programming, and C++ provides all required features (classes, inheritance, virtual functions, friend functions and pointers) in a direct and efficient way. It also runs quickly as a compiled console program and requires only the standard library, so no extra libraries had to be installed.

Contribution of the Concepts to the Application

- **Classes and objects** keep all data of one student together with the functions that operate on it.
- **Inheritance** avoids repeating ID and name by placing them in Person.
- **Function overloading** lets the grade be obtained either from a given percentage or directly from the student’s own marks.
- **Overriding and virtual functions** let a Person pointer display complete Student details.
- **Friend function** allows compare() to read the names of two students without extra getter functions.
- **Constructors** give safe default values, and **arrays and pointers** are used to store and traverse the records.

V. Code with Explanation

Program Structure

The whole project is written in a single file, main.cpp, in the following order:

- **Header files and namespace:** iostream, string and using namespace std.
- **Class Person:** base class holding the protected ID and Name, with a virtual display() and a virtual destructor.
- **Class Student:** derived class holding course, semester, subjects, marks and count, together with all result-related functions, the What-If Grade Calculator and the declaration of the friend function.
- **compare():** non-member friend function that compares two students.
- **searchStudent():** non-member function that finds a student by ID.
- **main():** creates the array of students and runs the menu loop.

The program does not contain a separate ResultManager class. The management work (adding, searching, deleting and so on) is done by the searchStudent() function and the switch cases in main(), and the diagram below follows the code as it actually is.

UML Class Diagram

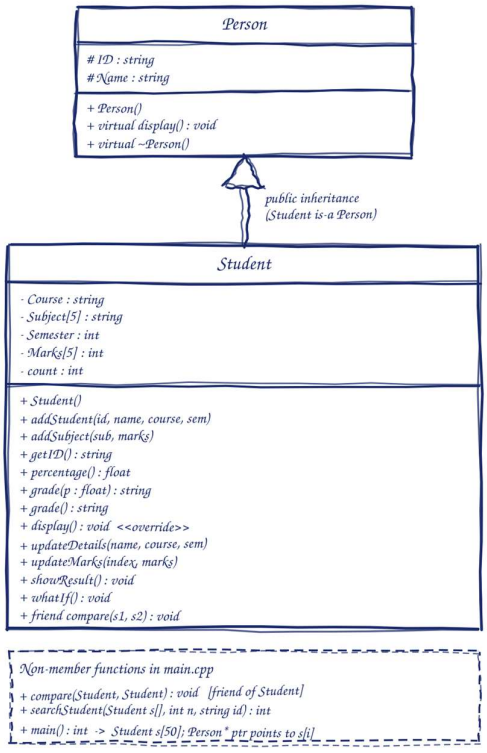


Figure 1: Hand-drawn UML class diagram showing Person, Student and their relationship

In the diagram, + means public, - means private and # means protected. The hollow triangle shows that Student publicly inherits from Person.

Complete Source Code (main.cpp)

```
#include <iostream>
#include <string>
using namespace std;

// Base Class
class Person {
protected:
    string ID, Name;

public:
    Person() {
        ID = "";
        Name = "";
    }

    virtual void display() {
        cout << "ID: " << ID << "\nName: " << Name << endl;
    }

    virtual ~Person() {}
};

// Derived Class
class Student : public Person {
private:
    string Course, Subject[5];
    int Semester, Marks[5], count;

public:
    Student() {
        Course = "";
        Semester = 0;
        count = 0;
    }

    void addStudent(string id, string name, string course, int sem) {
        ID = id;
        Name = name;
        Course = course;
        Semester = sem;
        count = 0;
    }

    void addSubject(string sub, int marks) {
        if (count < 5) {
            Subject[count] = sub;
            Marks[count] = marks;
            count++;
        }
    }

    string getID() {
        return ID;
    }

    // Function Overloading
    float percentage() {
        int total = 0;
        for (int i = 0; i < count; i++)
            total += Marks[i];

        return count == 0 ? 0 : (float)total / count;
    }

    string grade(float p) {
        if (p >= 90) return "A+";
```

```
        if (p >= 80) return "A";
        if (p >= 70) return "B";
        if (p >= 60) return "C";
        if (p >= 50) return "D";
        return "F";
    }

    string grade() {
        return grade(percentage());
    }

    // Function Overriding
    void display() override {
        Person::display();
        cout << "Course: " << Course
             << "\nSemester: " << Semester << endl;
    }

    void updateDetails(string name, string course, int sem) {
        Name = name;
        Course = course;
        Semester = sem;
    }

    void updateMarks(int index, int marks) {
        if (index >= 0 && index < count)
            Marks[index] = marks;
    }

    void showResult() {
        display();

        cout << "\nSubject\t\tMarks\n";

        for (int i = 0; i < count; i++)
            cout << Subject[i] << "\t\t" << Marks[i] << endl;

        cout << "\nPercentage: " << percentage() << "%";
        cout << "\nGrade: " << grade() << endl;
    }

    // Unique Feature: What-If Grade Calculator
    void whatIf() {
        if (count == 0) {
            cout << "No subjects available!\n";
            return;
        }

        for (int i = 0; i < count; i++)
            cout << i + 1 << ". " << Subject[i]
                 << " (" << Marks[i] << ")\n";

        int choice, newMarks;
        cout << "Select subject: ";
        cin >> choice;

        if (choice < 1 || choice > count) {
            cout << "Invalid choice!\n";
            return;
        }

        cout << "Enter hypothetical marks: ";
        cin >> newMarks;

        if (newMarks < 0 || newMarks > 100) {
            cout << "Invalid marks!\n";
            return;
```

```
        }

        float oldP = percentage();
        float newP = oldP + (newMarks - Marks[choice - 1]) / (float)count;

        cout << "\nOriginal Percentage: " << oldP << "%";
        cout << "\nPredicted Percentage: " << newP << "%";
        cout << "\nOriginal Grade: " << grade(oldP);
        cout << "\nPredicted Grade: " << grade(newP);
        cout << "\nChange: " << newP - oldP << "%\n";

        // Original marks are not modified.
    }

    // Friend Function
    friend void compare(Student s1, Student s2);
};

// Compare two students
void compare(Student s1, Student s2) {
    cout << "\n" << s1.Name << ": " << s1.percentage() << "%";
    cout << "\n" << s2.Name << ": " << s2.percentage() << "%\n";

    if (s1.percentage() > s2.percentage())
        cout << s1.Name << " has higher marks.\n";
    else if (s2.percentage() > s1.percentage())
        cout << s2.Name << " has higher marks.\n";
    else
        cout << "Both students have equal marks.\n";
}

// Search student by ID
int searchStudent(Student s[], int n, string id) {
    for (int i = 0; i < n; i++)
        if (s[i].getID() == id)
            return i;

    return -1;
}

// Main Function
int main() {
    Student s[50];
    int n = 0, choice, index;
    string id, name, course;
    int sem, subjects, marks, subChoice;

    do {
        cout << "\n===== STUDENT RESULT MANAGEMENT SYSTEM =====\n";
        cout << "1. Add Student\n";
        cout << "2. Display All Students\n";
        cout << "3. Search Student\n";
        cout << "4. Update Student Details\n";
        cout << "5. Update Marks\n";
        cout << "6. Performance Report\n";
        cout << "7. Delete Student\n";
        cout << "8. What-If Grade Calculator\n";
        cout << "9. Compare Students\n";
        cout << "10. Exit\n";
        cout << "Enter Choice: ";
        cin >> choice;

        switch (choice) {

        case 1:
            if (n == 50) {
                cout << "Student limit reached!\n";
```

```
            break;
        }

        cout << "Enter ID: ";
        cin >> id;

        if (searchStudent(s, n, id) != -1) {
            cout << "ID already exists!\n";
            break;
        }

        cout << "Enter Name (one word): ";
        cin >> name;
        cout << "Enter Course (one word): ";
        cin >> course;
        cout << "Enter Semester: ";
        cin >> sem;

        if (sem < 1 || sem > 12) {
            cout << "Invalid semester!\n";
            break;
        }

        s[n].addStudent(id, name, course, sem);

        cout << "Number of Subjects (1-5): ";
        cin >> subjects;

        if (subjects < 1 || subjects > 5) {
            cout << "Invalid subject count!\n";
            break;
        }

        for (int i = 0; i < subjects; i++) {
            string sub;
            cout << "Subject Name: ";
            cin >> sub;
            cout << "Marks: ";
            cin >> marks;

            if (marks < 0 || marks > 100) {
                cout << "Invalid marks! Student not added.\n";
                break;
            }

            s[n].addSubject(sub, marks);
        }

        if (s[n].percentage() >= 0 && subjects > 0 &&
            s[n].getID() == id) {
            n++;
            cout << "Student added successfully!\n";
        }
        break;

    case 2:
        for (int i = 0; i < n; i++) {
            Person* ptr = &s[i];
            ptr->display();
            cout << "Percentage: " << s[i].percentage() << "%\n";
            cout << "Grade: " << s[i].grade() << "\n\n";
        }
        if (n == 0) cout << "No records found!\n";
        break;

    case 3:
        cout << "Enter Student ID: ";
```

```
            cin >> id;
            index = searchStudent(s, n, id);

            if (index == -1)
                cout << "Student not found!\n";
            else
                s[index].showResult();
            break;

        case 4:
            cout << "Enter Student ID: ";
            cin >> id;
            index = searchStudent(s, n, id);

            if (index != -1) {
                cout << "New Name: ";
                cin >> name;
                cout << "New Course: ";
                cin >> course;
                cout << "New Semester: ";
                cin >> sem;

                if (sem >= 1 && sem <= 12) {
                    s[index].updateDetails(name, course, sem);
                    cout << "Details updated!\n";
                } else {
                    cout << "Invalid semester!\n";
                }
            } else cout << "Student not found!\n";
            break;

        case 5:
            cout << "Enter Student ID: ";
            cin >> id;
            index = searchStudent(s, n, id);

            if (index != -1) {
                cout << "Enter Subject Number: ";
                cin >> subChoice;
                cout << "Enter New Marks: ";
                cin >> marks;

                if (marks >= 0 && marks <= 100) {
                    s[index].updateMarks(subChoice - 1, marks);
                    cout << "Marks updated!\n";
                } else cout << "Invalid marks!\n";
            } else cout << "Student not found!\n";
            break;

        case 6:
            cout << "Enter Student ID: ";
            cin >> id;
            index = searchStudent(s, n, id);

            if (index != -1)
                s[index].showResult();
            else
                cout << "Student not found!\n";
            break;

        case 7:
            cout << "Enter Student ID: ";
            cin >> id;
            index = searchStudent(s, n, id);

            if (index != -1) {
                for (int i = index; i < n - 1; i++)
```

```
                    s[i] = s[i + 1];

                n--;
                cout << "Student deleted!\n";
            } else cout << "Student not found!\n";
            break;

        case 8:
            cout << "Enter Student ID: ";
            cin >> id;
            index = searchStudent(s, n, id);

            if (index != -1)
                s[index].whatIf();
            else
                cout << "Student not found!\n";
            break;

        case 9: {
            string id2;
            cout << "Enter First Student ID: ";
            cin >> id;
            cout << "Enter Second Student ID: ";
            cin >> id2;

            int i = searchStudent(s, n, id);
            int j = searchStudent(s, n, id2);

            if (i != -1 && j != -1)
                compare(s[i], s[j]);
            else
                cout << "Student not found!\n";
            break;
        }

        case 10:
            cout << "Thank you for using the system!\n";
            break;

        default:
            cout << "Invalid choice!\n";
        }

    } while (choice != 10);

    return 0;
}
```

Explanation of the Important Components

1. Header Files

iostream gives access to cin and cout for all input and output. **string** gives the string class, used for IDs, names, courses and subjects. The line using namespace std; avoids writing std:: before each name.

2. Person Class

Person is the base class. Its data members **ID** and **Name** are protected, so they cannot be accessed from outside but can be used by the derived Student class. The constructor sets both to empty strings. The function display() prints the ID and name and is declared **virtual**, and the destructor ~Person() is also virtual, which is the correct practice when a base class is meant to be used through pointers.

3. Student Class

Student is publicly derived from Person. Its private members are Course, the array Subject[5], Semester, the array Marks[5] and count. The constructor initialises Course to an empty string and Semester and

count to 0. addStudent() fills the ID, name, course and semester and resets count. addSubject() stores a subject name and its marks at position count and then increases count, but only while fewer than five subjects are stored. getID() returns the ID so that main() can compare IDs.

4. Result Calculation

percentage() adds the marks of the first count subjects in a loop and divides the total by count after converting it to float. If no subject exists (count is 0) it returns 0 to avoid division by zero. Every result shown by the program uses this function, so percentages are always consistent.

5. Grade Generation

The grade is produced by two functions with the same name but different parameters, which is **function overloading**. grade(float p) compares the given percentage against the boundaries 90, 80, 70, 60 and 50 and returns A+, A, B, C, D or F. grade() has no parameter; it calls grade(percentage()) so that a student's own grade can be obtained directly. (In the source, the comment "Function Overloading" appears just above percentage(), but the overloaded pair is the two grade() functions.)

6. Result Display

Student overrides display(): it first calls Person::display() to print the ID and name and then prints the course and semester. This is **function overriding**, and the keyword override lets the compiler check that a base class function is really being overridden. showResult() prints the details through display(), then a table of subjects and marks, and finally the percentage and grade.

7. Student Management

In main(), Student s[50] stores the records and n counts them. For Menu option 1 the program checks that the limit of 50 is not reached, that the ID is not already used, and that the semester is between 1 and 12 and the number of subjects between 1 and 5. It then asks for each subject name and marks (0 to 100) and calls addSubject(). Menu option 2 displays all records or prints "No records found!" when n is 0.

8. Search and Update Operations

searchStudent() is a linear search: it checks the ID of each of the n students and returns its index, or -1 if there is no match. Menu options 3, 4, 5, 6, 8 and 9 all begin with this search. updateDetails() replaces the name, course and semester. updateMarks() takes an index; main() passes the subject number minus 1, and the function ignores an index outside the stored subjects.

9. Delete Student Functionality

After finding the student, the program copies every following record one position to the left (s[i] = s[i + 1]) and reduces n by 1. The last record is thus overwritten and the deleted student disappears from all later operations.

10. Friend Function

compare() is declared inside Student with the keyword friend, but it is defined outside the class and is not a member. A friend function can use the private and protected data of the class, so compare() reads s1.Name and s2.Name directly. It prints both percentages and states which student has higher marks, or that both are equal. The students are passed by value, so the originals are not changed.

11. Runtime Polymorphism

In Menu option 2, the code Person* ptr = &s[i]; creates a base-class pointer that points to a Student object. Since display() is virtual, the call ptr->display() runs **Student::display()** and not Person::display(); the function is chosen at run time according to the real type of the object. This is runtime polymorphism.

12. Main Function and Menu System

main() declares the variables, then repeats a do-while loop that prints the menu and reads the choice. A switch statement has a case for each of the options 1 to 9, a case 10 that prints "Thank you for using the system!" and a default case for an invalid choice. The loop continues until the user chooses 10.

Unique Feature: What-If Grade Calculator

WHAT-IF GRADE CALCULATOR (ACADEMIC PERFORMANCE SIMULATOR) – Menu option 8, implemented by the member function whatIf() of the Student class.

1. Introduction

A basic result system only reports what has already happened. Students usually want to know what would happen if they scored better in one subject, for example "how much will my overall grade improve if I score 90 instead of 84 in SE?". The What-If Grade Calculator answers this question instantly. It was added so that the project is not only a record keeper but also a small decision-support tool, which makes it more useful than an ordinary Student Result Management System.

2. Working

1. The user selects Menu option 8 and enters the ID of a registered student; if the ID is not found, "Student not found!" is displayed.
2. The application displays the student's subjects with their actual marks, numbered from 1.
3. The user selects a subject by its number (an invalid number prints "Invalid choice!").
4. The user enters the hypothetical marks (a value outside 0–100 prints "Invalid marks!").
5. The application calculates the predicted overall percentage.
6. The application generates the predicted grade using the same grade(float) function.
7. The application displays the original percentage, predicted percentage, original grade, predicted grade and the change.

3. Calculation

Original Percentage = Original Total Marks / Number of Subjects

Predicted Percentage = (Original Total Marks − Original Subject Marks + Hypothetical Subject Marks) / Number of Subjects

Performance Change = Predicted Percentage − Original Percentage

The program uses an equivalent form: newP = oldP + (newMarks − Marks[choice − 1]) / count. Since oldP is the total divided by the count, adding the difference of the two marks divided by the count gives exactly the same value as the predicted-percentage formula, without recalculating the total.

4. Example

Consider a student with five subjects:

Table 5: Actual marks of the example student

Subject	Actual Marks
Python	85
DBMS	72
SE	84
OOPS	78
Comm Skills	80

Original Total = 85 + 72 + 84 + 78 + 80 = **399/500**; Original Percentage = 399 / 5 = **79.8%**; Original Grade = **B** (the percentage is below 80). Now assume the student wants to know what happens if the SE marks increase from 84 to 90:

Table 6: What-If simulation of the example (SE: 84 to 90)

Item	Value
Original SE Marks	84
Hypothetical SE Marks	90
Predicted Total	399 − 84 + 90 = 405/500
Predicted Percentage	405 / 5 = 81%
Original Grade	B
Predicted Grade	A
Performance Change	81 − 79.8 = +1.2 percentage points

A rise of only six marks in one subject lifts the student from grade B to grade A, because the new percentage crosses the boundary of 80. The same calculation is shown with real program output in Figure 11, where changing OOPS from 74 to 96 for a four-subject student raises the percentage from 86.25% to 91.75%, a change of 5.5, and the grade from A to A+.

5. Important Advantage

The hypothetical marks are stored only in the local variable newMarks and are used only for calculation. The Marks[] array is never assigned in whatIf() (the source contains the comment “Original marks are not modified”). Therefore the actual academic record stays unchanged, and the student can try as many different values as desired. Performance Report (Menu option 6) shows the original marks and percentage again after a simulation.

6. Significance

The feature helps a student understand how a single subject influences the overall result. Since every subject is worth an equal share, each mark in one subject changes the percentage by 1/count of a mark. The student can find out how much improvement is needed to reach the next grade, decide in which subject extra effort will be worthwhile, and see the effect of a possible drop in marks as well. Technically, it also shows how an existing member function (grade) can be reused for a new purpose and how a calculation can be done without changing stored data.

VI. Output

This section shows the actual screenshots of the program run, each followed by a short analysis. The run uses student IDs 0029 (entered as Som and later renamed Somya), 0030 (Darsh) and 0031 (Harsh, added only to demonstrate deletion). IDs are stored as strings, so the leading zeros are kept.

Output 1: Main Menu and Adding the First Student

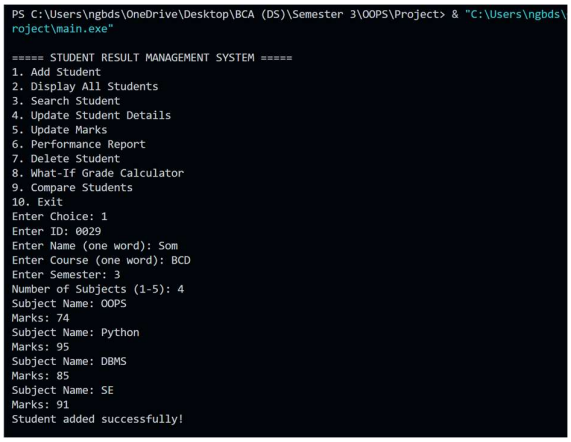


Figure 2: Main menu and adding the first student (ID 0029)

The program starts with the title and the ten menu options. Choice 1 is entered, followed by the ID, name, course, semester and four subjects. The message "Student added successfully!" confirms that the record was stored.

Analysis

Table 7: Data entered for student 0029

ID	Name	Subjects entered	Marks
0029	Som	OOPS, Python, DBMS, SE	74, 95, 85, 91

The ID, semester (within 1–12), subject count (within 1–5) and every mark (within 0–100) were accepted. The total of the four marks is 74 + 95 + 85 + 91 = 345.

Output 2: Adding the Second Student

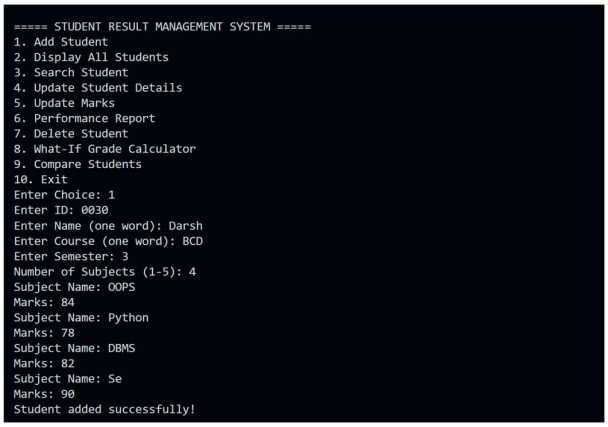


Figure 3: Adding the second student (ID 0030)

A second student, Darsh (ID 0030, course BCD, semester 3), is added with OOPS 84, Python 78, DBMS 82 and Se 90.

Analysis

The program checks that the ID 0030 is not already in use before accepting the record. The total of Darsh's marks is 84 + 78 + 82 + 90 = 334, which gives 334 ÷ 4 = 83.5%. The subject name was typed as "Se" and is stored exactly as entered.

Output 3: Displaying All Students

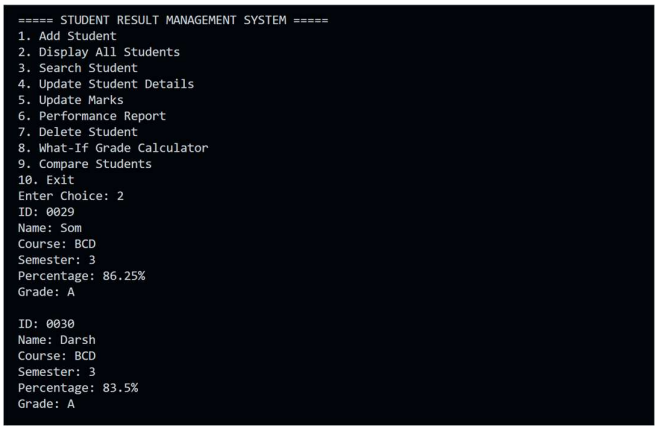


Figure 4: Display All Students (menu option 2)

Both records are listed with ID, name, course, semester, percentage and grade. The listing is produced through a Person pointer that calls the overridden display() function.

Analysis

Table 8: Calculation behind the displayed percentages

Student	Total marks	Percentage	Grade	Rule
Som	74 + 95 + 85 + 91 = 345	345 ÷ 4 = 86.25%	A	80 to 89
Darsh	84 + 78 + 82 + 90 = 334	334 ÷ 4 = 83.5%	A	80 to 89

Output 4: Searching for a Student

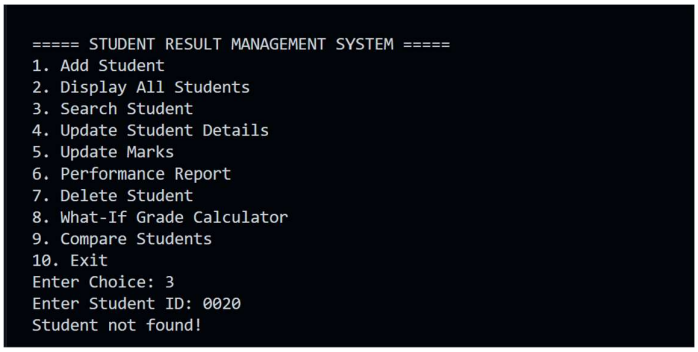


Figure 5: Search Student (menu option 3) with an ID that does not exist

The ID 0020 is entered. Since no stored student has this ID, searchStudent() returns -1 and the program prints "Student not found!".

Analysis

This screenshot shows the case of an unknown ID, so it confirms that the program does not crash when the search fails. When the ID exists, the same menu option prints the complete result using showResult(), in the format shown in Figure 8.

Output 5: Updating Student Details

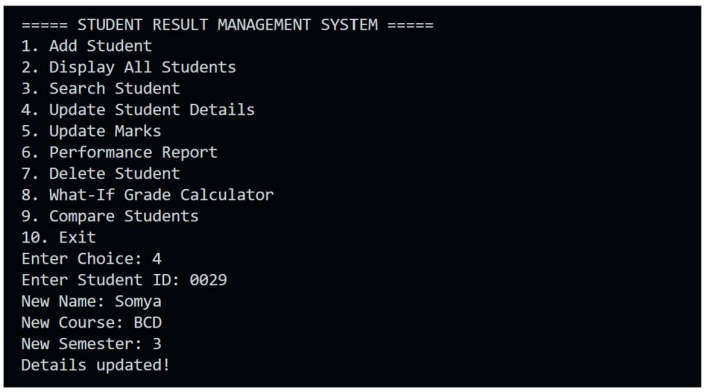


Figure 6: Update Student Details (menu option 4)

The ID 0029 is entered and the new name (Somya), course (BCD) and semester (3) are given. The message "Details updated!" confirms that updateDetails() changed the record.

Analysis

Table 9: Record 0029 before and after the update

Field	Before	After
Name	Som	Somya
Course	BCD	BCD
Semester	3	3

Only the name changed. The marks were not touched, and the new name appears in the performance report (Figure 8) and in the comparison output (Figure 12).

Output 6: Updating Subject Marks

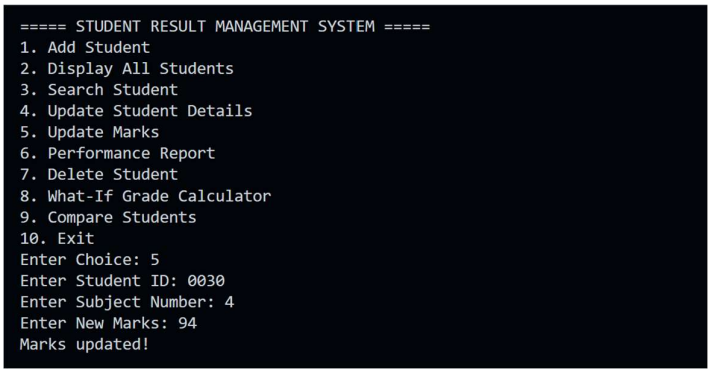


Figure 7: Update Marks (menu option 5) for student 0030

The ID 0030 is entered, subject number 4 is selected and the new marks 94 are given. The message "Marks updated!" confirms that updateMarks() changed the mark.

Analysis

Table 10: Effect of the mark update on student 0030

Item	Before	After	Subject
Marks in subject 4	90	94	Se
Total marks	334	338	–
Percentage	83.5%	84.5%	–

The subject number entered by the user is reduced by 1 inside main() because the array index starts at 0. The new percentage, (84 + 78 + 82 + 94) ÷ 4 = 84.5%, is the value later printed by the comparison in Figure 12.

Output 7: Performance Report

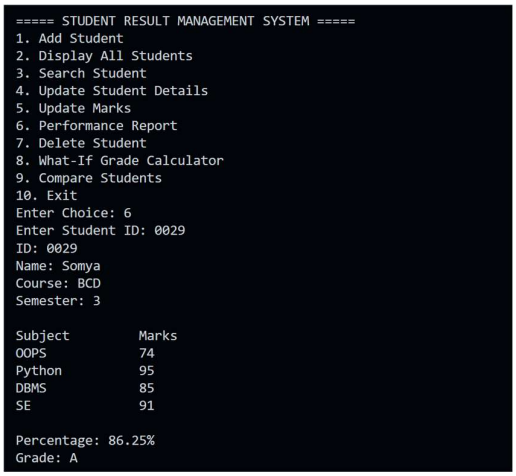


Figure 8: Performance Report for ID 0029 (menu option 6)

The report prints the student's details, a table of the four subjects and their marks, the percentage (86.25%) and the grade (A).

Analysis

The report shows the updated name Somya, and the percentage 345 ÷ 4 = 86.25% matches the value in the earlier listing (Figure 4), because both use percentage() and grade(). Her highest mark is in Python (95) and her lowest in OOPS (74).

Output 8: Adding a Third Student for the Delete Demonstration

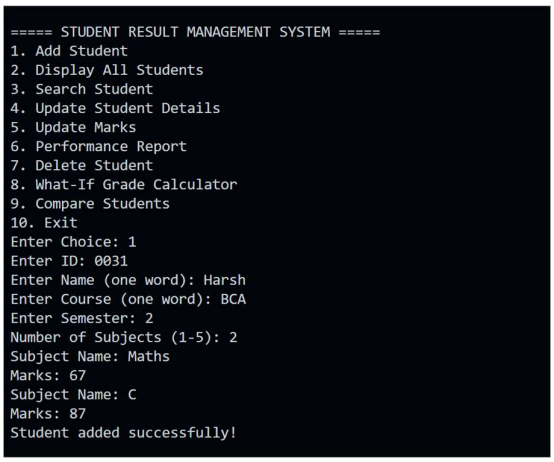


Figure 9: Adding student 0031 (Harsh) with two subjects

Harsh (ID 0031, course BCA, semester 2) is added with two subjects, Maths 67 and C 87.

Analysis

This record shows that a student can have fewer than five subjects: the percentage is calculated over the number of subjects actually entered, (67 + 87) ÷ 2 = 77%. The record is added so that it can be deleted in the next output.

Output 9: Deleting a Student

```
===== STUDENT RESULT MANAGEMENT SYSTEM =====
1. Add Student
2. Display All Students
3. Search Student
4. Update Student Details
5. Update Marks
6. Performance Report
7. Delete Student
8. What-If Grade Calculator
9. Compare Students
10. Exit
Enter Choice: 7
Enter Student ID: 0031
Student deleted!
```

Figure 10: Delete Student (menu option 7) for ID 0031

The ID 0031 is entered and the program prints “Student deleted!”.

Analysis

The record is found with searchStudent(), the later records are shifted one position to the left and the student counter is reduced by 1. Harsh no longer takes part in any later operation, which is why the comparison and the summary table contain only Somya and Darsh.

Output 10: What-If Grade Calculator

```
===== STUDENT RESULT MANAGEMENT SYSTEM =====
1. Add Student
2. Display All Students
3. Search Student
4. Update Student Details
5. Update Marks
6. Performance Report
7. Delete Student
8. What-If Grade Calculator
9. Compare Students
10. Exit
Enter Choice: 8
Enter Student ID: 0029
1. OOPS (74)
2. Python (95)
3. DBMS (85)
4. SE (91)
Select subject: 1
Enter hypothetical marks: 96

Original Percentage: 86.25%
Predicted Percentage: 91.75%
Original Grade: A
Predicted Grade: A+
Change: 5.5%
```

Figure 11: What-If Grade Calculator (menu option 8) for ID 0029

The subjects are listed with their actual marks: OOPS (74), Python (95), DBMS (85) and SE (91). Subject 1 (OOPS) is selected and 96 is entered as hypothetical marks. The program shows the original and predicted percentage and grade, and the change.

Analysis

Table 11: Original and predicted result for Somya (OOPS: 74 to 96)

Item	Original	Predicted
OOPS marks	74	96 (hypothetical)
Total marks	345	367
Percentage	86.25%	91.75%
Grade	A	A+
Change	–	+5.5

The predicted percentage agrees with the formula: 86.25 + (96 − 74) ÷ 4 = 91.75. The simulated mark lifts the percentage across the 90 boundary, so the grade would change from A to A+. The stored OOPS mark remains 74, as the Performance Report in Figure 8 would still show, because the simulation does not modify the record.

Output 11: Comparing Two Students

```
===== STUDENT RESULT MANAGEMENT SYSTEM =====
1. Add Student
2. Display All Students
3. Search Student
4. Update Student Details
5. Update Marks
6. Performance Report
7. Delete Student
8. What-If Grade Calculator
9. Compare Students
10. Exit
Enter Choice: 9
Enter First Student ID: 0030
Enter Second Student ID: 0029

Darsh: 84.5%
Somya: 86.25%
Somya has higher marks.
```

Figure 12: Compare Students (menu option 9)

The IDs 0030 and 0029 are entered. The friend function compare() prints both percentages and states which student has higher marks.

Analysis

compare() printed Darsh: 84.5% and Somya: 86.25%, and concluded that Somya has higher marks. Darsh's value is the updated percentage from Output 6, and the name Somya comes from the update in Output 5.

Output 12: Exiting the Application

```
===== STUDENT RESULT MANAGEMENT SYSTEM =====
1. Add Student
2. Display All Students
3. Search Student
4. Update Student Details
5. Update Marks
6. Performance Report
7. Delete Student
8. What-If Grade Calculator
9. Compare Students
10. Exit
Enter Choice: 10
Thank you for using the system!
PS C:\Users\ngbds\OneDrive\Desktop\BCA (DS)\Semester 3\OOPS\Project>
```

Figure 13: Exit (menu option 10)

Choice 10 is entered; the program prints "Thank you for using the system!" and returns to the terminal prompt.

Analysis

Choosing 10 ends the do-while loop in main(), and the program finishes with return 0. All records held in memory are lost at this point, as explained in the limitations in Section III.

Overall Result Summary

Table 12: Final result of the students at the end of the run

ID	Name	Total	Percentage	Grade
0029	Somya	345	86.25%	A
0030	Darsh	338	84.5%	A

The totals and percentages are the values after the updates in Outputs 5 and 6. Student 0031 was deleted in Output 9.

VII. Conclusion

The project successfully delivers a working console-based Student Result Management System in C++. A user can add students with up to five subjects, view all records, search by ID, update details and marks, print individual performance reports, delete records and compare two students. The outputs in Section VI match the calculations done by hand, which confirms that the percentage and grade logic works correctly.

The system simplifies result management because totals, percentages and grades are produced automatically and in the same way for every student, and because any record can be located immediately by its ID. This removes the repeated calculation and searching effort of a manual register.

The project applies the OOPS concepts in a practical way: a Person base class and a student derived class (inheritance), encapsulated data, constructors and a virtual destructor, overloaded grade() functions, an overridden display() function, a virtual function called through a base-class pointer (runtime polymorphism) and a friend function for comparison.

The What-If Grade Calculator is the most distinctive part of the project. It lets a student test hypothetical mark in a subject and see the predicted percentage, grade and change without altering the real record, so the application helps in planning and not only in recording.

Limitations and possible improvements. The data is lost when the program closes, so a future version could save records in files or a database. Other improvements are accepting names with spaces, validating non-numeric input, handling an invalid mark during student entry properly, allowing more than five subjects, adding a separate manager class for the records, and extending the What-If Calculator to change several subjects at once.

VIII. Learning Outcomes

Developing this project taught me both the theory and the practical use of Object-Oriented Programming. The main things I learned are given below.

- **C++ programming:** I learned how a complete C++ program is organised, how to use control statements, loops and functions, and how to design a menu-driven program with do-while and switch.
- **Classes and objects:** I understood how data and functions are combined in a class and how each student record becomes an object, including an array of objects.
- **Constructors and destructors:** I saw how constructors give objects safe starting values, and why a base class used polymorphically needs a virtual destructor.
- **Inheritance and encapsulation:** I learned to place common data in a base class, to inherit it in Student, and to protect data with protected and private access.
- **Function overloading and overriding:** I understood the difference between two grade() functions with different parameters and a derived-class display() that replaces the base-class version.
- **Virtual functions and polymorphism:** I learned that a base-class pointer can call the derived-class version of a virtual function, which I used in Display All Students.
- **Pointers:** I practised using a Person pointer that holds the address of a Student object.
- **Friend functions:** I learned how a non-member function can be given access to a class's private data and when this is useful, as in compare().
- **Arrays and strings:** I worked with arrays of strings, integers and objects and with the string class, including deleting an element by shifting the array.
- **Menu-driven applications and input validation:** I learned to validate the ID, semester, subject count and marks to keep the program stable.
- **Problem-solving and logical thinking:** Designing the percentage, grade and What-If calculations, and finding the limits of my own code, such as the invalid-marks case, improved my logical thinking and testing skills.

IX. GitHub Link

GitHub Repository: https://github.com/Somya-ds/Student-Result-Management-System

Repository Name: Student-Result-Management-System